

Projekt 1

Simon Buschmann, Yannik Buchner - LLLF(MLU)

Nikita Podibko, Jan Draeger - ALU

Johannes Heinrich, Malek Haoues Rhaïem - APL

TU Dortmund

Inhaltsverzeichnis

1 Least Loaded Link First (MLU)

2 Average Link Utilization

3 Average Path Length

Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link First (MLU)

Least Loaded Link
First (MLU)

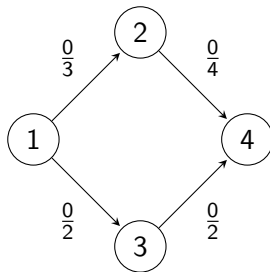
Average Link
Utilization

Average Path
Length

- Betrachte jeden Auftrag nacheinander einzeln.
- Nutze Dijkstra um für diesen Auftrag einen Pfad von s nach t zu bestimmen, der die Utilisation minimal hält.
- Behandle dabei nicht Knoten mit minimaler Distanz, sondern mit minimale Utilisation auf Pfad.

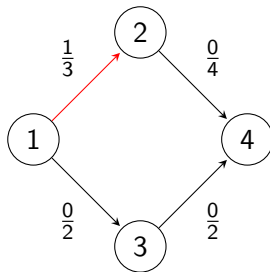
Beispiel

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$



Beispiel

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$



Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

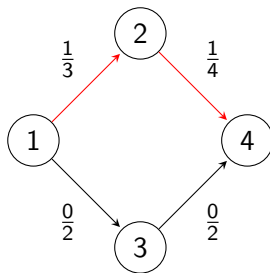
Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Beispiel

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$



Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

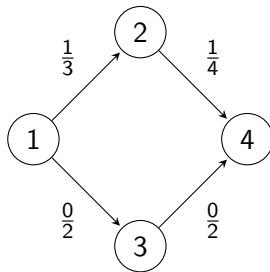
Average Link
Utilization

Average Path
Length

Beispiel - Fortsetzung

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$

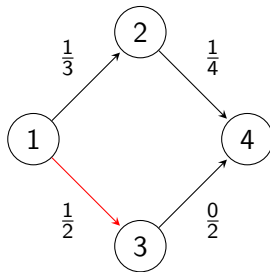
Auftrag 2: $s = 1$, $t = 4$, $demand = 1$



Beispiel - Fortsetzung

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$

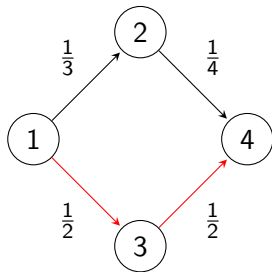
Auftrag 2: $s = 1$, $t = 4$, $demand = 1$



Beispiel - Fortsetzung

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$

Auftrag 2: $s = 1$, $t = 4$, $demand = 1$



Algorithm 1 LLLFSolver

Require: $G(V, E, c : E \mapsto \mathbb{R}), D = \{D_1 = (s_1, t_1, d_1), D_2, \dots, D_n\}$

Ensure: $P = \{P_1, P_2, \dots, P_n\}$

```
1: for  $e \in E$  do
2:    $loads_e \leftarrow 0$ 
3: end for
4:  $P \leftarrow \{\}$ 
5: for  $i \in \{1, \dots, n\}$  do
6:    $P_i \leftarrow PotentialUtilizationDijkstra(G, loads, D_i)$ 
7:   for  $e \in P_i$  do
8:      $loads_e \leftarrow loads_e + d_i$ 
9:      $utilization_e \leftarrow loads_e / c(e)$ 
10:  end for
11:   $P \leftarrow P \cup \{P_i\}$ 
12: end for
13: return  $P$ 
```

Algorithm 2 PotentialUtilizationDijkstra

Require: $G(V, E, c : E \mapsto \mathbb{R}), loads, D_i = (s_i, t_i, d_i)$

Ensure: P_1

```
1: for  $v \in V$  do
2:    $utilization_v \leftarrow \infty$ 
3:    $predecessor_v \leftarrow -1$ 
4:    $visited_v \leftarrow \text{False}$ 
5: end for
6:  $utilization_{s_i} \leftarrow 0$ 
7: while  $visited_{t_i} == \text{False}$  do
8:    $u \leftarrow v$  with minimal  $utilization_v$ 
9:   for  $n \in neighbours(u)$  do
10:    if  $utilization_n > \max\{utilization_u, (loads_{(u,n)} + d_i)/c(u, n)\}$  then
11:       $predecessor_n \leftarrow u$ 
12:       $utilization_n \leftarrow \max\{utilization_u, (loads_{(u,n)} + d_i)/c(u, n)\}$ 
13:    end if
14:  end for
15:   $visited_u \leftarrow \text{True}$ 
16: end while
17:  $P_1 \leftarrow$  Generate Path with  $predecessor$  starting at  $predecessor_t$ 
18: return  $P_1$ 
```

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Average Link Utilization

Gründe für die Betrachtung der Average Link Utilization

- Gibt ein vollständigeres Bild über die allgemeine Ressourcennutzung.
- Unterstützt die effiziente Nutzung der verfügbaren Kapazitäten.
- Ermöglicht die Erkennung von Überlastungen und Unterauslastungen.
- Liefert einen aussagekräftigen Wert über die Verteilung der Lasten im Netzwerk.
- Weist auf die Gesamtnutzung der Netzressourcen hin und zeigt mögliche Optimierungsansätze auf, die mit MLU allein nicht sichtbar wären.

Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Vor- und Nachteile von ALU im Vergleich zu MLU

Vorteile von Average Link Utilization (ALU)

- Besseres Gesamtbild: Erfasst die durchschnittliche Nutzung aller Links.
- Effizienz sichtbar: Zeigt, wie gut das Netzwerk insgesamt ausgelastet ist.
- Optimierungspotenzial: Erkennt Unterauslastung und unausgenutzte Ressourcen.
- Vergleichbarkeit von Algorithmen: Kleine Unterschiede in Lastverteilung werden erkennbar.

Nachteile von ALU gegenüber MLU

- **Verdeckt Engpässe:** Einzelne überlastete Links können durch Durchschnitt verdeckt werden.
- **Keine Worst-Case-Aussage:** Kritische Belastungssituationen bleiben unentdeckt.
- **Trügerische Werte:** Niedrige ALU trotz schlechter Lastverteilung möglich.

Vorgehensweise zur Berechnung der Average Link Utilization

- 1 Code zur Berechnung der Average Link Utilization (ALU) erstellen.
- 2 Diesen Code in die Factory-Klasse integrieren.
- 3 Aufruf der Berechnung in `main.py` einfügen.
- 4 Visualisierung der Ergebnisse (z. B. mittels Matplotlib).

Formel

$$ALU = \frac{1}{|L|} \sum_{l \in L} \frac{f_l}{c_l}$$

Legende

- L : Menge aller Links im Netzwerk
- $|L|$: Anzahl der Links
- f_l : Verkehrsmenge (Flow) auf Link l
- c_l : Kapazität des Links l
- $\frac{f_l}{c_l}$: Auslastung (Utilization) des Links l

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Average Path Length

Motivation für APL als Zielfunktion

- Effiziente Wege führen zu kurzer Latenz und schneller Kommunikation
- Geringerer Ressourcenverbrauch was zu geringeren Kosten führt
- Das Average Path Length ist ein einfach interpretierbares Maß
- Die Pfadlänge optimieren als Design-Maß für Netzwerke

Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Mathematische Definition APL

Formel

$$APL = \frac{1}{\sum_{i=1}^{|D|} d_i} \sum_{(s_i, t_i, d_i) \in D} \text{len}(P_i) \cdot d_i$$

Legende:

- D : Menge aller Demands der Form (s_i, t_i, d_i)
- s_i : Startknoten des i -ten Demands
- t_i : Zielknoten des i -ten Demands
- d_i : Gewicht des i -ten Demands
- P_i : Pfad vom Quellknoten $s - i$ zum Zielknoten t_i
- $\text{len}(P_i)$: Anzahl der Hops im Pfad P_i

Algorithm 3 Berechne Average Path Length (APL)

Require: $D = \{(S_i, T_i, d_i)\}$, Pfadlängenfunktion $\text{len}(P_i)$

Ensure: APL-Wert

```
1: totalWeightedPathLength  $\leftarrow 0$ 
2: totalDemandWeight  $\leftarrow 0$ 
3: for jedes  $(s_i, t_i, d_i) \in D$  do
4:   if  $s_i \neq t_i$  and  $P_i$  exists then
5:      $P_i \leftarrow \text{berechnePfad}(s_i, t_i)$ 
6:     length  $\leftarrow \text{len}(P_i)$ 
7:     totalWeightedPathLength  $\leftarrow \text{totalWeightedPathLength} + \text{length} \cdot d_i$ 
8:     totalDemandWeight  $\leftarrow \text{totalDemandWeight} + d_i$ 
9:   else
10:    continue
11:  end if
12: end for
13: APL  $\leftarrow \text{totalWeightedPathLength} / \text{totalDemandWeight}$ 
14: return APL
```

Nachteile der APL

- Nicht robust bei isolierten Komponenten
- Nicht für Vermeidung von Engpässen geeignet
- APL ist ein Mittelwert → Anfällig für Ausreißer
- hohe Komplexität

Projekt 1

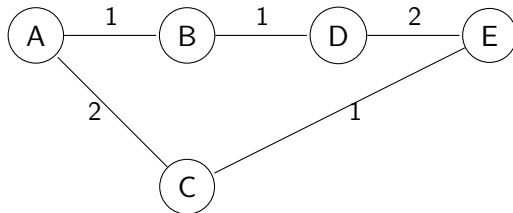
Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Beispiel APL



Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoques
Rhaïem - APL

Least Loaded Link
First (MLU)

Average Link
Utilization

Average Path
Length

Beispiel APL

Tabelle: Pfade, Hops, Gewichtung

Von	Nach	Demand D	Kürzester Pfad	Hops	$D \times H$
A	B	1	$A \rightarrow B$	1	1
A	C	2	$A \rightarrow C$	1	2
A	D	1	$A \rightarrow B \rightarrow D$	2	2
A	E	1	$A \rightarrow C \rightarrow E$	2	2
B	D	1	$B \rightarrow D$	1	1
B	E	1	$B \rightarrow D \rightarrow E$	2	2
C	E	1	$C \rightarrow E$	1	1
D	E	2	$D \rightarrow E$	1	2

$$\text{APL} = \frac{\sum(\text{Hops} \times D)}{\sum D} = \frac{1 + 2 + 2 + 2 + 1 + 2 + 1 + 2}{1 + 2 + 1 + 1 + 1 + 1 + 1 + 2} = \frac{13}{10} = 1.3$$